



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A Project Proposal
On
Math2Visual: From Natural Language Prompts to Manim Code for Animated Video
Generation**

Submitted By:

Ashish Kandel (THA078BEI006)
Binita Adhikari (THA078BEI008)
Mukesh Bhatta (THA078BEI022)
Nischal Bhusal (THA078BEI024)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

May, 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, for providing us with the opportunity to undertake this project as part of the requirements for the Bachelor's degree in **Electronics, Communication and Information Engineering**.

We are especially thankful to our respected Head of Department, **Er. Umesh Kanta Ghimire** sir, for his continuous support and encouragement throughout the proposal phase of this project.

We would also like to extend our heartfelt thanks to **Er. Sudip Rana** sir, the Project Coordinator, and **Er. Prabin Sharma Poudel** sir for their valuable guidance, feedback, and support during the development of the project proposal.

Working on this project will not only strengthen our technical knowledge but also improve our collaboration and problem-solving skills. We look forward to learning and growing throughout the course of this project.

Ashish Kandel (THA078BEI006)

Binita Adhikari (THA078BEI008)

Mukesh Bhatta (THA078BEI022)

Nischal Bhusal (THA078BEI024)

ABSTRACT

The Math2Visual: From Natural Language Prompts to Manim Code for Animated Video Generation project focuses on transforming the way mathematical concepts are taught and understood by bridging natural language input with animated visualizations. The system takes plain-language descriptions such as “plot the graph of $y = \sin(x)$ ” and automatically converts them into Manim code to generate corresponding animations. By leveraging technologies like Natural Language Processing (NLP), AI-driven code generation, and symbolic math parsing, the project makes it easier for educators and learners to create intuitive and dynamic visual content without needing advanced programming skills. This tool aims to make complex math topics more accessible, especially in digital learning environments, by automating the creation of explanatory animations from user-friendly prompts.

Keywords: *AI-powered learning, Attention mechanism, Code generation, Educational animation, Explainable AI, Fine-tuning, Large language models (LLMs), Manim, Math visualization, Natural language processing (NLP), Neural networks, Open-source AI, Prompt engineering, Python automation, Text-to-code translation, Text-to-video, Transformer models, Visualization tools*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Definition	2
1.4 Objectives	2
1.5 Scope of Project	3
1.6 Application of Project	3
1.7 Limitations	4
2 LITERATURE REVIEW	5
3 REQUIREMENT ANALYSIS	9
3.1 Hardware Requirements	9
3.1.1 High Performance GPU	9
3.2 Software Requirements	9
3.2.1 Python	9
3.2.2 PyTorch	9
3.2.3 Transformers Library (Hugging Face)	10
3.2.4 Manim (Mathematical Animation Engine)	10
3.2.5 CUDA Toolkit and cuDNN (GPU Drivers)	10
3.2.6 Jupyter Notebook or Visual Studio Code (VS Code)	10
3.2.7 FFmpeg	10
3.2.8 Kaggle Notebooks	11
3.2.9 Pandas and NumPy	11
3.2.10 Matplotlib / Seaborn	11
3.2.11 Git and GitHub	11
4 METHODOLOGY	12
4.1 Overview	12

4.2	Dataset Preparation and Model Fine-Tuning	12
4.3	System Architecture	13
4.3.1	Natural Language Input	13
4.3.2	Text Understanding (Text Encoder - LLM)	13
4.3.3	Semantic Parser and NL to Code Mapper	14
4.3.4	Code Generator Module (Generates Manim Code)	14
4.3.5	Code Validator and Syntax Checker	14
4.3.6	Manim Renderer Engine	14
4.3.7	Output Video File	14
4.4	Tools and Technologies Component	15
4.5	GUI Implementation	15
4.5.1	Python GUI	15
4.6	Evaluation and Testing Strategy	15
4.6.1	Code Accuracy	16
4.6.2	Animation Correctness	16
4.6.3	Latency	16
4.6.4	Testing Metrics	17
4.6.5	Test Cases	17
5	FEASIBILITY ANALYSIS	19
5.1	Economic Feasibility	19
5.2	Technical Feasibility	19
5.3	Operational Feasibility	19
5.4	Ethical Feasibility	19
6	EXPECTED OUTCOME	20
6.1	Prompt vs. Generated Code Example	20
6.1.1	Input Prompt	20
6.1.2	Mathematical Interpretation	20
6.1.3	Generated Manim Code	20
6.1.4	Generated video after compilation of code	22
6.1.5	Wireframe	23
7	APPENDICES	24
	Appendix A: Gantt Chart	24
	Appendix B: Expected Budget	25
	Appendix C: Mathematical Foundation of Transformer-Based LLMs	26

List of Figures

Figure 4-1: Proposed System Block diagram	13
Figure 6-1: Rendered output of the animation generated from prompt	22
Figure 6-2: Wireframe of the Math2Visual interface showing the code generation panel	23
Figure 6-3: Wireframe of the Math2Visual interface displaying the video playback panel	23
Figure 7-1: Gantt chart	24

List of Tables

Table 4-1: Tools and Technologies Component	15
Table 4-2: Example Test Cases for Evaluation	18
Table 7-1: Budget Estimation of the project	25

List of Abbreviations

3D	Three-Dimensional
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CV	Computer Vision
DL	Deep Learning
DNN	Deep Neural Network
HF	Hugging Face
GPT	Generative Pre-training Transformer
GPU	Graphical Processing Unit
LLM	Large Language Model
Manim	Mathematical Animation Engine
ML	Machine Learning
NL2Code	Natural Language to Code
NLP	Natural Language Processing
NLU	Natural Language Understanding
NN	Neural Network
RLHF	Reinforcement Learning with Human Feedback
SVG	Scalable Vector Graphics
UI	User Interface

1. INTRODUCTION

Mathematics is a foundational subject in engineering, science, and technology, but many students find it difficult to understand because of its abstract nature and complex symbols. Traditional teaching methods often fail to help students build a clear mental picture of mathematical ideas. To solve this problem, our project aims to develop an AI-powered system that takes a math-related sentence written in natural language and converts it into Manim code. When this code is compiled, it produces an animated video that visually explains the given concept. This system will help students learn math more easily by turning difficult equations or descriptions into clear, engaging animations.

1.1. Background

Mathematical visualization plays a crucial role in enhancing comprehension, problem-solving, and retention. Recent educational studies emphasize that learners, particularly in STEM fields, benefit significantly from graphical and animated representations of abstract concepts. Tools like Manim (Mathematical Animation Engine) have emerged as powerful platforms for creating high-quality animations of mathematical phenomena. However, Manim's steep learning curve and its dependency on Python scripting limit its accessibility to educators and learners without programming experience.

In parallel, advancements in Natural Language Processing (NLP) and large language models (LLMs) have enabled systems to understand and generate human-like text, including executable code. These technologies can serve as a bridge between user-friendly language input and complex backend tools like Manim. By leveraging NLP and AI-driven code generation, it is now feasible to create systems that understand a user's intent and automatically produce the corresponding animation scripts, reducing the technical burden and enabling a wider audience to visualize mathematical ideas effectively.

1.2. Motivation

Even though many online platforms and tools exist for learning mathematics, very few can turn mathematical ideas into dynamic animations without needing programming skills. Most systems require users to manually write animation scripts or depend on static images and pre-recorded videos, which often don't match the exact concept being taught. This makes it difficult for both students and educators to visualize important topics such as functions, geometric shapes, algebraic transformations, and calculus-based change especially when those topics involve continuous motion or change.

Traditionally, math visualizations are created using tools like GeoGebra, Desmos, or by manually coding animations using engines like Manim. While these tools are powerful, they either lack flexibility in animation or require technical knowledge that not all learners or teachers possess. As a result, much time is spent either learning to code or finding pre-made content, which limits creativity and accessibility in classrooms.

This project is inspired by the idea of making mathematics more interactive and intuitive using the power of artificial intelligence. By allowing users to type math-related prompts in plain English and automatically generating Manim code that produces visual animations, we aim to remove the technical barrier. This approach brings together natural language processing (NLP) and programmatic visualization, creating a smart, user-friendly tool that can assist learners, teachers, and content creators alike.

In a world where technology is rapidly transforming education, this project offers a new way to engage with mathematical content visually, helping to bridge the gap between symbolic math and conceptual understanding. It has the potential to improve math learning outcomes, especially for students who struggle with abstract thinking or who learn better through visual means.

1.3. Problem Definition

Understanding mathematics can be difficult for many learners, especially when it involves abstract ideas and complex symbols. Most existing math learning tools rely heavily on static images, formulas, and text-based explanations. These methods often do not provide the visual clarity that students need to fully grasp mathematical concepts. Moreover, manually creating visual animations for math content requires programming skills, time, and effort, which is not practical for all learners or educators.

There is currently no widely available system that allows users to simply type math-related ideas in natural language and automatically receive meaningful animated visualizations. This lack of intuitive tools makes it harder for students to build a strong conceptual foundation in mathematics and slowly lose the interest in subject as well.

1.4. Objectives

The main objectives of proposed project are:

1. To develop a Natural Language Processing module that can accurately interpret mathematical prompts written in plain English and convert them into corresponding Manim code.

2. To implement a system that compiles the generated Manim code and renders it as an animated video to visually represent the described mathematical concept.

1.5. Scope of Project

This project focuses on developing an AI-powered software system that transforms plain English descriptions of mathematical concepts into animated visualizations. The system will utilize natural language processing techniques to interpret user input and automatically generate Manim (Mathematical Animation Engine) code. This code will then be compiled to create animations that help visually convey the underlying mathematical ideas. The project will support a range of fundamental mathematical topics such as geometry, algebra, equations, and graph-based concepts. It will feature a user-friendly interface that allows users to input natural language prompts and view the resulting animations. The backend will handle language understanding, code generation, and rendering, forming a complete pipeline from text to animation. By combining NLP and visual representation through animation, the system aims to make learning mathematics more intuitive, engaging, and accessible, especially for students and educators in academic settings.

1.6. Application of Project

1. **Interactive Math Learning** : Transforms written math descriptions into animations, making abstract topics easier to visualize and grasp for students.
2. **Smart Content Creation for Educators** : Helps teachers and tutors quickly generate custom visualizations for lectures, tutorials, and presentations using simple English instructions.
3. **Test Preparation and Concept Review Tools** : Used in test-prep apps (e.g., SAT, GRE, JEE) to generate on-the-fly visual explanations for practice problems or review sessions.
4. **Integration with AR/VR Learning Environments** : The generated Manim outputs could be extended into augmented or virtual reality settings to immerse learners in 3D mathematical environments.

1.7. Limitations

1. The system supports only basic mathematical topics like geometry, algebra, simple graphs, and equations. Advanced topics such as calculus or differential equations are not included.
2. Natural language processing works best with clear and simple English prompts; ambiguous or complex sentences may cause incorrect Manim code generation.
3. The system currently supports only English text input and does not handle multiple languages or voice commands.

2. LITERATURE REVIEW

Mathematics education significantly benefits from visualization, as it bridges the gap between abstract concepts and intuitive understanding. Visualization, especially through animations, has proven effective in enhancing students grasp of complex mathematical concepts by offering dynamic representations that static images and texts often fail to convey [1]. Animated visuals can portray intricate mathematical transformations, spatial relationships, and functions dynamically, thus improving comprehension, engagement, and retention among learners [2]. This pedagogical effectiveness underscores the need for accessible, intuitive tools that facilitate math animations without requiring extensive technical knowledge from educators or students. .

Currently, several frameworks and tools exist for mathematical visualization, notably Manim, GeoGebra, and Desmos. Manim, developed initially by Grant Sanderson (3Blue1Brown), is highly praised for producing professional-quality math animations used extensively in educational YouTube videos [3]. However, despite its powerful capabilities, Manim’s accessibility remains limited due to its steep learning curve, as it requires proficiency in Python programming and familiarity with its scripting style [4]. GeoGebra and Desmos offer user-friendly graphical interfaces but generally lack the dynamic animation scripting capabilities that Manim provides. Thus, there exists a clear gap between ease of use and the production of detailed, custom animated visuals.

Advances in natural language processing (NLP) have enabled substantial improvements in educational technology interfaces. Symbolic mathematics libraries, like SymPy, combined with NLP, can interpret human language descriptions into structured symbolic expressions or mathematical operations [5]. Furthermore, tools leveraging large language models (LLMs) such as GPT, Code LLaMA, and Phi have shown promising results in translating natural language queries into executable mathematical code [6]. For instance, the integration of OpenAI’s GPT models with mathematical engines allows users to formulate complex mathematical questions in natural language, significantly reducing barriers for those without strong programming backgrounds [7].

Large language models have increasingly gained traction in automating code generation tasks, substantially impacting software engineering practices. Models like GPT-4, LLaMA, Mistral, Phi-2, and Falcon exhibit strong performance in generating syntactically correct and contextually accurate code snippets from textual descriptions [8]. Research by Chen et al. on OpenAI's Codex demonstrated remarkable capabilities in Python code synthesis, significantly outperforming earlier models in tasks involving converting natural language into executable code [9]. Likewise, Meta's Code LLaMA and Microsoft's Phi-2 have set benchmarks in open-source code generation, proving their utility in real-world software development scenarios [10]. These advances demonstrate the practicality of employing LLMs in automated mathematical visualizations, indicating potential integration with animation libraries such as Manim.

Several studies have explored translating natural language into executable mathematical code. Hendrycks et al. presented transformer-based methods for code generation tasks, specifically emphasizing Python and JavaScript, showcasing that transformer models significantly outperform previous rule-based or template-driven approaches [11]. Similarly, research efforts leveraging Hugging Face Transformers highlighted effective fine-tuning techniques for generating domain-specific mathematical code [12].

However, relatively few studies directly connect natural language prompts with complex mathematical animation scripts. Initial attempts to bridge this gap often resulted in either limited scope or reliance on predefined templates, thereby reducing generalizability and flexibility [13]. For instance, existing tools might translate simple phrases to basic animations but fail to handle intricate mathematical instructions or provide meaningful interactivity or adaptability to user needs.

Fareed Khan's project, AI Text to Video Model from Scratch, offers a foundational approach to generating video content directly from textual prompts using deep learning techniques. Built entirely in Python, the model utilizes Generative Adversarial Networks (GANs) to translate descriptive sentences into corresponding video sequences. The work involves essential stages such as data preprocessing, text embedding using Word2Vec, and constructing a GAN-based architecture to handle the generation process. Although the system is lightweight and intended for educational purposes, it effectively demonstrates the feasibility of aligning textual semantics with visual content generation. This project provides valuable insights into the core mechanics of AI-powered video generation and serves as a practical reference for developing systems like Math2Visual, which aim to transform natural language math descriptions into automated, educational animations [14].

The Text2Scene model by Tan et al. [15] introduces a sequential method for generating visual scenes from text by predicting and placing objects with specific attributes like size, position, and appearance. Unlike end-to-end image generators, it offers interpretable, compositional outputs an approach that aligns well with the structured nature of educational animations. For Math2Visual, this method supports the idea of breaking down mathematical prompts into individual visual elements, making it a relevant reference for text-to-animation systems.

OpenAI’s model, Sora, presents a diffusion transformer architecture capable of generating high-definition, minute-long videos directly from textual descriptions. According to OpenAI’s documentation and technical details, Sora focuses on maintaining temporal consistency and simulating realistic physical interactions, although it still faces challenges in modeling complex causal reasoning and real-world physics [16].

Similarly, Google DeepMind’s Veo model represents a multimodal generative system that produces 1080p videos of over a minute duration. As outlined in recent public documentation, Veo integrates audio with video and allows fine-grained editing. Its third version (released in 2025) includes synchronized audio capabilities, illustrating the potential of aligning language, visual scenes, and audio streams in educational video generation [17]. Dream Machine by Luma AI introduces a lightweight solution capable of producing five-second videos from either text or image prompts. It emphasizes motion realism and camera perspective simulation. While praised for its smooth rendering of human gestures and cinematic movement, critiques have emerged regarding its lack of transparency in training data and model source details [18].

The reviewed literature reveals notable limitations in current approaches, especially in automating the generation of professional-quality animations directly from natural language descriptions without extensive coding expertise. Although NLP and LLMs demonstrate potential in simplifying code generation, their application in specialized math animation domains remains underexplored. Additionally, the existing animation frameworks, notably Manim, have not been effectively integrated with NLP and LLM-based code generation methodologies in an accessible, user-friendly interface.

Furthermore, comparative evaluations of multiple LLMs in terms of animation quality, code correctness, and execution efficiency in generating complex mathematical animations are scarce. This gap indicates an opportunity for comprehensive research and practical implementation to enhance user accessibility and productivity in educational technology.

Building upon the identified research gaps, the proposed ”Math2Visual” project aims to develop a lightweight, intuitive system leveraging open-source LLMs (e.g., Code LLaMA, Mistral, Phi-2) to translate natural language prompts directly into executable Manim anima-

tions. This project uniquely addresses the accessibility issue by providing a Python GUI interface (via Tkinter or PyQt), facilitating interaction for users lacking programming skills. By systematically comparing multiple LLM outputs based on accuracy, rendering quality, and response efficiency, this project aspires to set a practical benchmark for future developments in AI-assisted educational visualization tools.

3. REQUIREMENT ANALYSIS

This section outlines fundamental hardware and software prerequisites essential for the successful execution and completion of this project.

3.1. Hardware Requirements

3.1.1. High Performance GPU

A high-performance GPU is crucial for fine-tuning large language models like Code LLaMA and Mistral, which contain billions of parameters. These models require immense parallel processing power that is only feasible with modern GPUs. GPUs significantly reduce the training time, allow larger batch sizes, and support operations like attention mechanisms and matrix multiplications at high speed. Without a capable GPU, the model training would be extremely slow or even infeasible. Additionally, GPU acceleration is necessary during inference to provide fast, real-time code generation based on user prompts.

3.2. Software Requirements

3.2.1. Python

Python is the backbone of this project, providing the primary language for all development work. It enables seamless integration of machine learning libraries (like PyTorch and Hugging Face Transformers) and visualization tools (like Manim). The Python ecosystem also includes utility libraries for data manipulation, training visualization, and prompt preprocessing. Version 3.10 or higher is recommended for compatibility with the latest features in these frameworks.

3.2.2. PyTorch

PyTorch is the deep learning framework used to fine-tune and evaluate large language models in this project. It supports GPU acceleration, dynamic computational graphs, and rich debugging capabilities. PyTorch is compatible with the Hugging Face Transformers library, enabling straightforward integration of LLMs like Code LLaMA and Mistral. It is widely adopted in the research community and known for its flexibility and ease of use in both training and inference stages.

3.2.3. Transformers Library (Hugging Face)

The Hugging Face Transformers library provides access to pre-trained language models, tokenizers, and training utilities. It simplifies the process of fine-tuning and evaluating models using high-level APIs. The library also includes tools for managing datasets, tracking training progress, and exporting models for deployment. It forms the core of the model interaction layer in this project, enabling input prompts to be processed and corresponding code to be generated.

3.2.4. Manim (Mathematical Animation Engine)

Manim is a Python-based engine used to create animated mathematical visualizations. In this project, it converts the model-generated Python code into explanatory videos. Manim allows frame-by-frame control of animations, supports LaTeX for mathematical notation, and includes a wide range of classes for geometrical and graphical visualization. It bridges the gap between abstract code and visual learning by producing professional-quality educational animations from user prompts.

3.2.5. CUDA Toolkit and cuDNN (GPU Drivers)

For GPU acceleration to work with PyTorch and Hugging Face models, the CUDA Toolkit and cuDNN drivers must be installed. These tools provide the low-level hardware support needed to utilize the full computational power of NVIDIA GPUs. They ensure compatibility between software libraries and GPU hardware, enabling efficient parallel computation, matrix multiplication, and deep learning operations on massive datasets.

3.2.6. Jupyter Notebook or Visual Studio Code (VS Code)

These integrated development environments (IDEs) are suitable for writing, testing, and debugging code. Jupyter is ideal for rapid prototyping and experimentation, while VS Code offers a full development suite with Git integration and debugging tools. Both environments support live visualization, markdown annotations, and plugins for Python, Manim, and AI development, making them ideal for iterative development in this project.

3.2.7. FFmpeg

FFmpeg is required by Manim to encode and export animations into video formats such as MP4 or GIF. It handles tasks like frame rendering, compression, audio syncing, and encoding with high efficiency. Without FFmpeg, Manim cannot compile its rendered frames into final

video output. This makes FFmpeg essential for converting the visualized Manim scenes into educational video clips.

3.2.8. Kaggle Notebooks

Kaggle provides a free, cloud-based environment with GPU support, making it ideal for model training and experimentation when local resources are limited. It supports collaborative workflows, public datasets, and version tracking via notebooks, aiding reproducibility and benchmarking.

3.2.9. Pandas and NumPy

These libraries are essential for preprocessing datasets, handling structured inputs/outputs, and computing metrics during training and evaluation. They streamline tasks such as prompt formatting, result comparison, and loss tracking.

3.2.10. Matplotlib / Seaborn

These visualization tools help plot training losses, evaluation metrics, and model comparison graphs. They provide insights into model performance and help guide future fine-tuning iterations.

3.2.11. Git and GitHub

Version control using Git and online repositories like GitHub helps collaborate with team, track code changes and manage training experiments. It is especially helpful for storing model checkpoints, code versions, and training logs. GitHub also provides issue tracking, continuous integration, and a platform for sharing your final project with the community or team members.

4. METHODOLOGY

4.1. Overview

This project proposes an AI-powered pipeline that converts natural language prompts into executable Manim code and renders the corresponding animated mathematical videos. The system utilizes fine-tuned large language models (LLMs) to automate code generation and embeds this process into a user-friendly Python-based graphical interface. The goal is to enhance accessibility to mathematical visualization without requiring programming knowledge.

4.2. Dataset Preparation and Model Fine-Tuning

To enable accurate conversion of natural language to Manim code, a custom dataset of prompt–code pairs will be prepared. This dataset will consist of :

- Manually curated examples of mathematical topics (e.g., graphing, equations, transformations) and their corresponding Manim code.
- Synthetic examples generated using templates and variations to expand coverage.
- Validation and test sets to evaluate generalization across math topics and input phrasing.
- Multiple open-source LLMs including LLaMA, Mistral, and Phi-2 will be fine-tuned using these datasets. The fine-tuning will focus on code generation performance using frameworks like Hugging Face Transformers and DeepSpeed. Models will be evaluated on:
 1. **Code correctness:** Whether the generated code runs without errors.
 2. **Response time:** Inference speed under identical conditions.
 3. **Animation quality:** Clarity, completeness, and accuracy of rendered output.

4.3. System Architecture

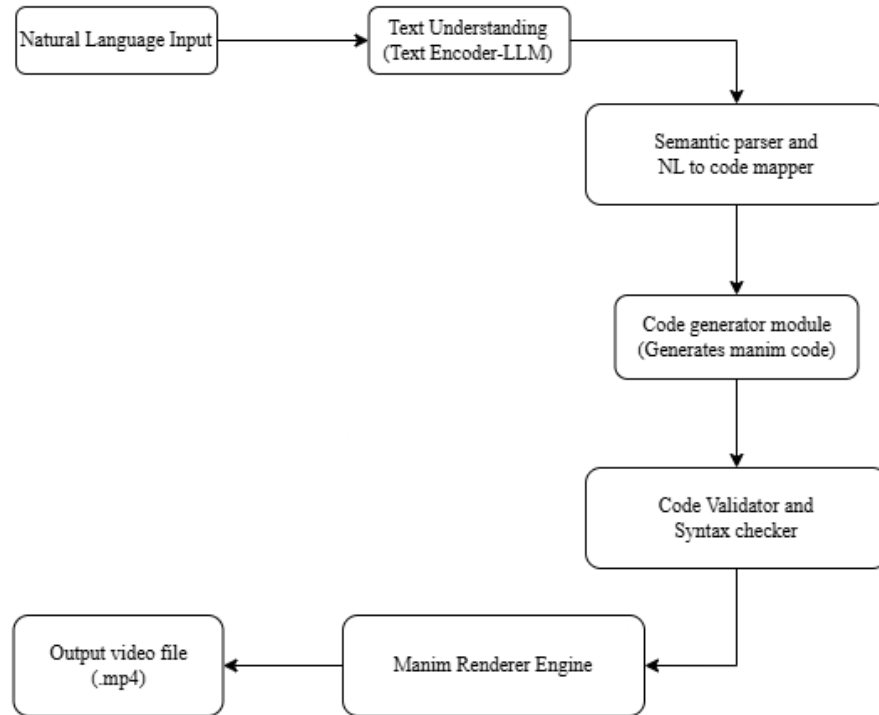


Figure 4-1: Proposed System Block diagram

The proposed system contains following components:

4.3.1. Natural Language Input

The process starts with the user entering a description in natural language, typically representing a mathematical concept or a visual idea. Examples are given in form of “draw a circle” or “show how matrix multiplication works.” This simple input interface does not need any programming knowledge. It is the foundation for converting high-level intent to animated visual data.

4.3.2. Text Understanding (Text Encoder - LLM)

This module uses a transformer-based encoder or large language model (LLM) to comprehend the input. It analyzes the sentence, identifies entities, actions, and relationships, and constructs a useful internal representation. Essentially, it translates human intent into structured data. Downstream parsing accuracy relies significantly on this step.

4.3.3. Semantic Parser and NL to Code Mapper

The semantic parser converts the structured knowledge to logical code units. It forms associations between natural language keywords and patterns to existing Manim structures. The module serves as a concept-to-code translator. It serves as the foundation for code generation.

4.3.4. Code Generator Module (Generates Manim Code)

In this, the mapped items are used to produce valid Manim Python code. The module auto-generates animations based on templates or dynamic generation approaches. It conceals syntax and API complexity from the user. The output is ready-to-render Manim script.

4.3.5. Code Validator and Syntax Checker

Ensures the generated code is syntactically correct and adheres to the structure required by Manim.

4.3.6. Manim Renderer Engine

This section takes the code that was created and converts it into a visual animation. The Manim library is utilized to create high-quality vector-based frames. The engine handles exact timing, transitions, and graphics.

4.3.7. Output Video File

This is the final product: a video animation that mathematically visualizes the math concept. It may be used in educational content, slideshows, or tutoring websites. The video is optimal as unambiguous, uniform, and consistent with the initial user demand. These products can be downloaded, replayed, or integrated into educational materials by students.

4.4. Tools and Technologies Component

Table 4-1: Tools and Technologies Component

Components	Tools/Technologies
LLM finetuning	Hugging face transformers, Pytorch
Code Generation Models	Phi-2, LLaMa, Mistral
Visualization Engine	Manim
GUI framework	Tkinter/PyQt/Gradio
File and Video Handling	OS/File APIs, ffmpeg, Shutil
Evaluation Metrics	Accuracy, Runtime, Visual Quality

4.5. GUI Implementation

A desktop GUI will be developed using Tkinter, PyQt, or Gradio (desktop mode), offering the following features:

- Code output area.
- Animation video player.
- Option to copy or edit or export code.

4.5.1. Python GUI

We used python GUI instead of traditional frontend and backend due to :

- i. Reduction of deployment complexity.
- ii. Enables full offline functionality.
- iii. Simplifies integration with Manim (already Python-based).
- iv. Offers lightweight and fast interaction.

4.6. Evaluation and Testing Strategy

Evaluation is a critical component of the system development process, especially for applications involving artificial intelligence, natural language processing, and automated code

generation. For the *Math2Visual* system, it is essential to assess not only whether the generated Manim code is syntactically and semantically correct, but also whether the rendered animation aligns with the user’s intended mathematical concept. This section outlines the strategies and metrics adopted to rigorously test the system’s functionality, reliability, and usability.

4.6.1. Code Accuracy

This metric evaluates whether the generated Manim code:

- Compiles successfully without syntax or runtime errors.
- Follows correct Manim syntax conventions.
- Produces the intended animation without requiring manual correction.

Each generated code snippet is automatically parsed and tested in a sandbox environment before animation rendering. Logs are maintained to track compilation success rates and error types.

4.6.2. Animation Correctness

Even if code compiles, it is essential to verify whether the output animation visually reflects the meaning of the input prompt. Evaluation is conducted by:

- Human reviewers comparing input prompts to rendered animations.
- A checklist evaluating visual correctness (e.g., proper axes, labels, curve shape, annotation).
- Rating animations based on fidelity to the intended mathematical concept.

4.6.3. Latency

System responsiveness is another key factor, particularly in educational tools. Latency is measured in two stages:

1. **Inference Latency:** Time taken by the LLM to generate the Manim code.
2. **Rendering Latency:** Time required by the Manim engine to produce the final video (e.g., MP4 or GIF).

These timings are recorded using timestamps and averaged across test cases.

4.6.4. Testing Metrics

To quantify the quality of code and animation generation, the following evaluation metrics are employed:

- **BLEU Score:** BLEU (Bilingual Evaluation Understudy) measures how closely the generated code matches a reference (ground truth) code. It compares n-gram overlaps (e.g., matching sequences of tokens) between generated and reference code. A higher BLEU score means the generated code is more similar in structure and content to the ideal output. This metric is commonly used in machine translation and adapted for code generation evaluation.
- **Edit Distance:** Evaluates the number of character-level or token-level changes needed to convert generated code into correct code.
- **Execution Success Rate:** This measures the percentage of generated code samples that run successfully without syntax or runtime errors. It tests whether the code is syntactically valid and functionally executable. A higher execution success rate implies that the model generates usable and error-free code more reliably. It's a practical indicator of code quality beyond textual similarity

4.6.5. Test Cases

A range of test prompts were used to validate system performance. Below are selected examples:

Table 4-2: Example Test Cases for Evaluation

Prompt (Input)	Expected Output (Math)	Success/Failure
Plot the graph of $y = \sin(x)$ from $-\pi$ to π	Smooth sine wave curve, labeled axes	Success
Visualize the unit circle with angle $\theta = \frac{\pi}{4}$ marked	Circle with radius 1 and an angle line showing $\frac{\pi}{4}$	Success
Draw $y = \frac{1}{x}$ and highlight vertical asymptote at $x = 0$	Hyperbola, vertical line at $x = 0$	Partial Success (asymptote not rendered)
Animate the derivative of $y = x^2$	Show original curve and its tangent moving across x-axis	Success
Show integral of x^2 from 0 to 2	Curve $y = x^2$ and shaded area under curve from 0 to 2	Success

Each test case is reviewed for both technical correctness and visual clarity. Notes are maintained to log partial errors, such as missing labels, incorrect axes, or incomplete animations.

5. FEASIBILITY ANALYSIS

5.1. Economic Feasibility

The project is economically viable, as it leverages open-source tools such as Manim, Hugging Face LLMs, and Python libraries, eliminating licensing costs. Development can be carried out using personal hardware, requiring minimal additional investment.

5.2. Technical Feasibility

The system is technically feasible with the current availability of open-source large language models (LLMs) and visualization tools. Python GUI frameworks like Tkinter or PyQt, along with Manim CE, can be integrated on mid-range computing systems commonly available to students.

5.3. Operational Feasibility

The application is designed to be user-friendly and accessible for students and educators, enabling intuitive interaction through plain English prompts. The system meets its operational goals by delivering accurate animated outputs with minimal user effort, promoting efficient math learning.

5.4. Ethical Feasibility

The project is ethically sound as it uses open-source tools and publicly available models, ensuring transparency and responsible use of AI. No personal or sensitive data is collected during use. The system is designed for educational purposes only, helping users learn mathematical concepts through visual explanations without risk of misuse or bias.

6. EXPECTED OUTCOME

The successful implementation of the **Math2Visual: From Natural Language Prompts to Manim Code for Animated Video Generation** project is expected to result in a system capable of accurately translating natural language prompts related to mathematical concepts into executable Manim code. By fine-tuning open-source large language models such as Code LLaMA and Mistral, the system aims to generate high-quality Python scripts that can be automatically rendered into educational animations. This will significantly reduce the time, effort, and technical knowledge traditionally required to create math visualizations, making it accessible for both educators and learners with limited programming skills.

Furthermore, the project will enable detailed comparative analysis of different LLMs based on training loss, code quality, and output effectiveness. The system's performance will provide insights into the practical viability of fine-tuned models in domain-specific code generation tasks. Ultimately, this research is expected to contribute to the growing field of AI-powered educational technologies by offering a tool that simplifies and enhances the teaching and learning of complex mathematical topics through dynamic, visual content.

6.1. Prompt vs. Generated Code Example

The following example illustrates a basic prompt processed by the *Math2Visual* system and the corresponding Manim code generated to create a simple animation. When the output code from the model is compiled, it generates the mathematical animated video according to the prompt provided as shown in the figure below.

6.1.1. Input Prompt

"Plot the graph of $y = x^2$ and mark the vertex."

6.1.2. Mathematical Interpretation

$$y = x^2 \quad \text{with vertex at } (0, 0)$$

6.1.3. Generated Manim Code

```
from manim import *

class SimpleParabola(Scene):
```

```

def construct(self):
    # Create basic 2D axes
    ax = Axes(
        x_range=[-5, 5, 1],
        y_range=[-5, 10, 1],
        axis_config={"color": WHITE}
    )

    # Add axis labels using Text() instead of LaTeX
    x_label = Text("x").scale(0.5).next_to(ax.x_axis.get_end(), RIGHT)
    y_label = Text("y").scale(0.5).next_to(ax.y_axis.get_end(), UP)

    # Plot the function  $y = x^2$ 
    parabola = ax.plot(lambda x: x**2, color=BLUE)

    # Mark the vertex at (0, 0)
    vertex = Dot(ax.coords_to_point(0, 0), color=RED)
    vertex_label = Text("Vertex").scale(0.4).next_to(vertex, DOWN)

    # Animate the scene
    self.play(Create(ax))
    self.play(Write(x_label), Write(y_label))
    self.play(Create(parabola))
    self.play(FadeIn(vertex), Write(vertex_label))
    self.wait(2)

```

This output demonstrates the system's ability to convert a natural language prompt into a working Manim animation script. The graph of $y = x^2$ is plotted, the axes are labeled using plain text, and the vertex at the origin is marked with a red dot and a textual label.

6.1.4. Generated video after compilation of code

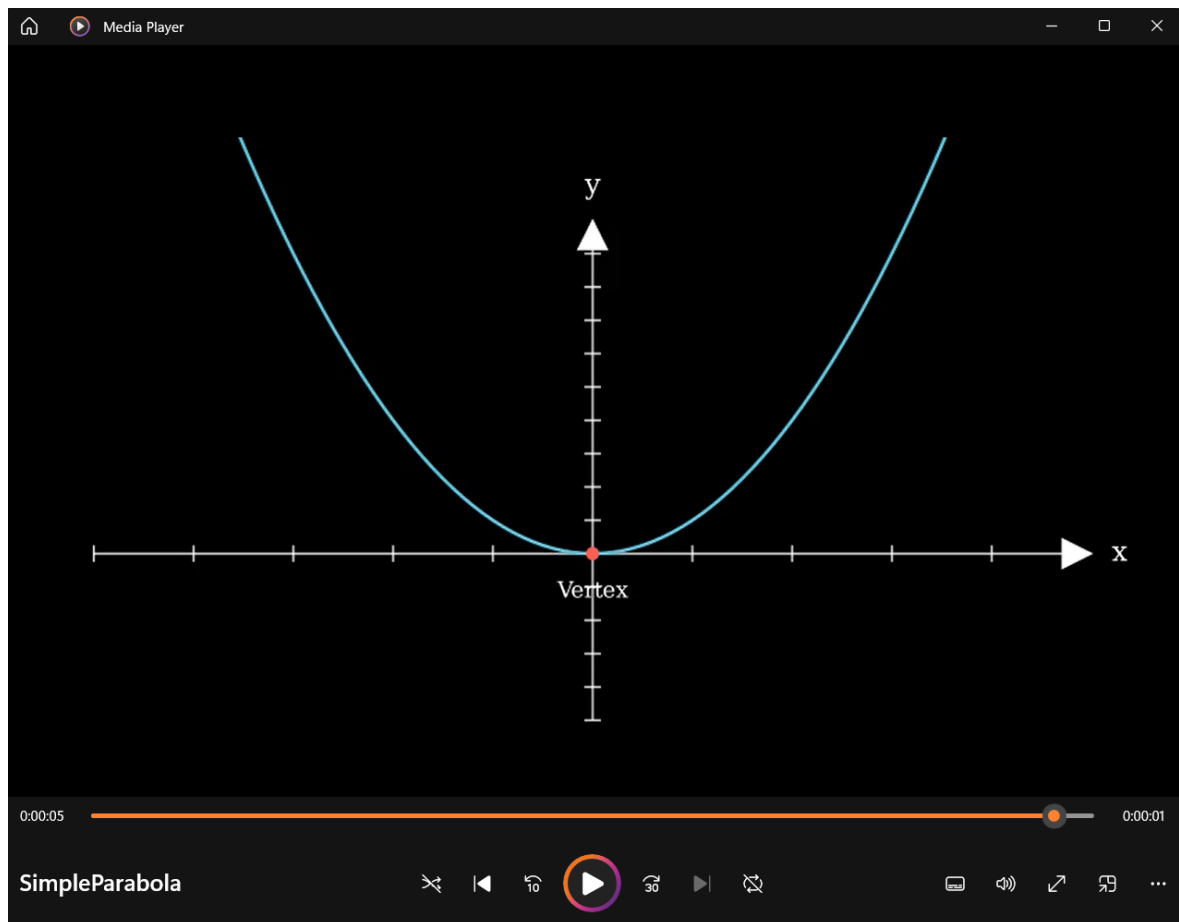


Figure 6-1: Rendered output of the animation generated from prompt

6.1.5. Wireframe

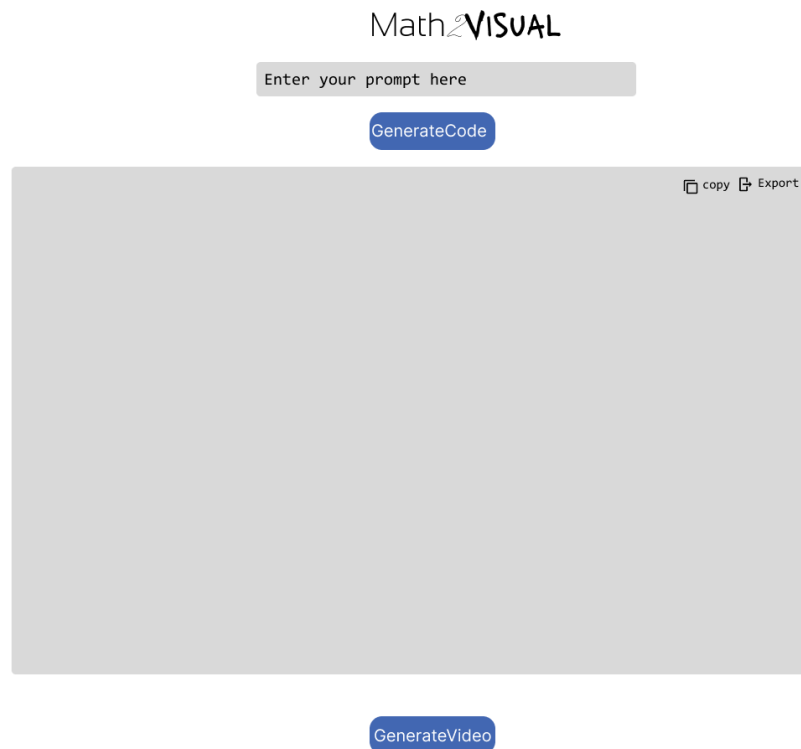


Figure 6-2: Wireframe of the Math2Visual interface showing the code generation panel

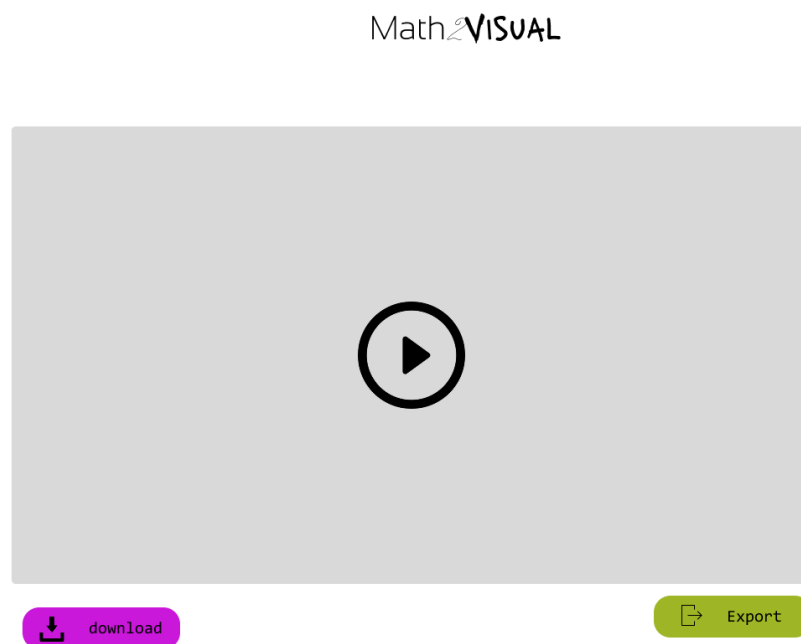


Figure 6-3: Wireframe of the Math2Visual interface displaying the video playback panel

7. APPENDICES

Appendix A: Gantt Chart

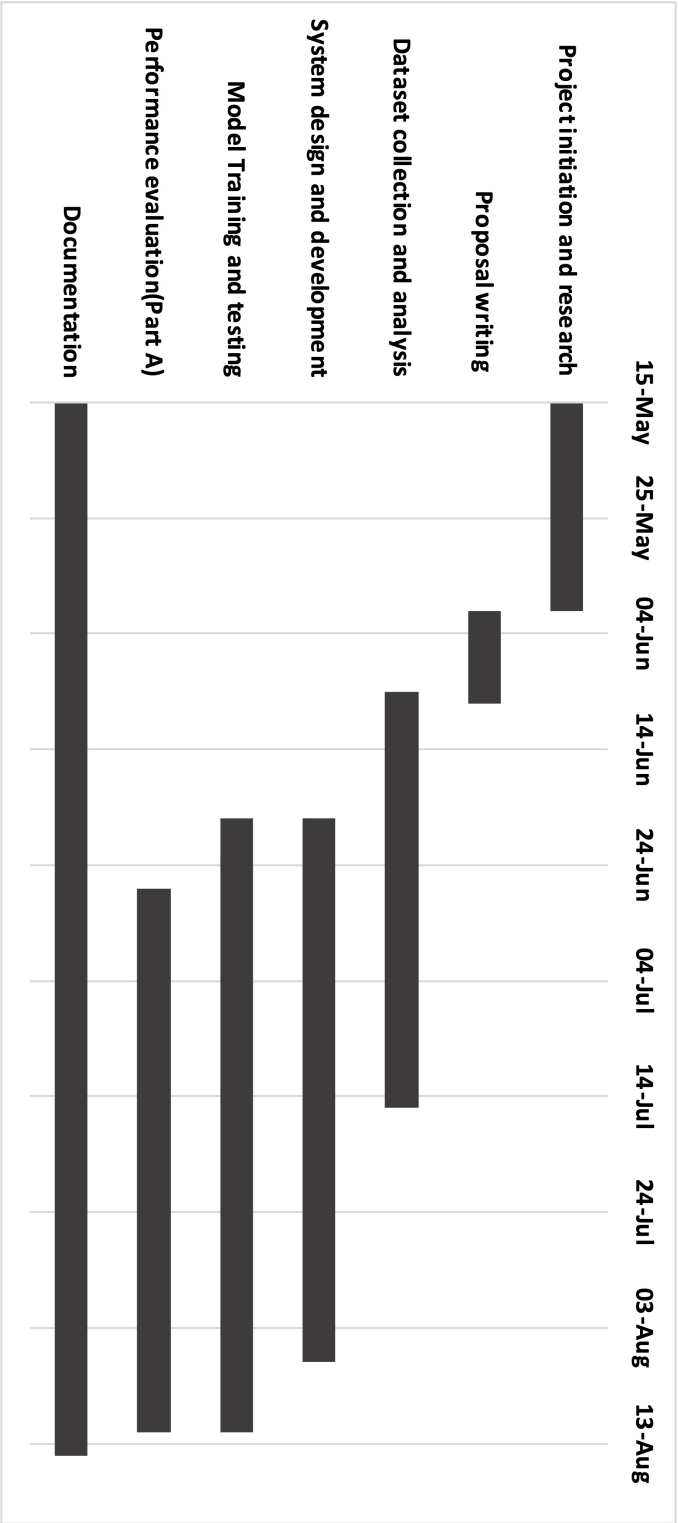


Figure 7-1: Gantt chart

Appendix B : Expected Budget

Table 7-1: Budget Estimation of the project

Components	Pricing (in NRs)
Google collab Pro (For GPU Access)	10,000
Cloud Storage (Hugging face or Open source LLM hosting)	5,000
Documentation	2,500
Miscellaneous	2,500
Total	20,000

Appendix C : Mathematical Foundation of Transformer-Based LLMs

To achieve the core objective of this project: translating natural language math prompts into animated Manim code, we fine-tune large language models (LLMs) such as Code LLaMA and Mistral. These models are built on the Transformer architecture, which relies heavily on linear algebra, calculus, and probability for both training and inference. This section outlines the key mathematical concepts and operations that govern how these models process and generate sequences, forming the basis for their fine-tuning in our project.

1. Token Embedding

$$\mathbf{E}_{\text{token}} = \text{Embedding}(x_i) \in \mathbb{R}^{d_{\text{model}}} \quad (1)$$

Each input token x_i is converted into a dense vector representation. This transformation allows the model to handle textual data in a mathematical format suitable for neural computation.

2. Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

Since Transformers are not inherently sequential, positional encoding is added to token embeddings to retain the order of words or symbols. These sinusoidal encodings allow the model to distinguish the position of tokens in a sentence.

3. Query, Key, and Value Projections

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (3)$$

Each input embedding X is linearly projected into three different spaces: Query, Key, and Value. These vectors are used in the attention mechanism to compute relevance between tokens.

4. Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4)$$

This mechanism enables the model to weigh the importance of each token in a sequence with respect to others. The dot product between queries and keys is scaled and passed through a softmax function to obtain attention weights, which are applied to the values.

5. Multi-Head Attention

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (5)$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Instead of computing attention once, the model performs it multiple times in parallel, each with different learned projections. This allows it to capture different types of relationships and features within the input sequence.

6. Feedforward Neural Network

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (6)$$

After attention, each token is passed through a two-layer feedforward neural network. This non-linear transformation enriches the token representations before passing to the next layer.

7. Softmax Output Layer

$$P(y_i|x) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (7)$$

At the output layer, the model computes a probability distribution over the vocabulary using the softmax function. The highest-probability token is selected during inference as the predicted next token.

8. Cross-Entropy Loss

$$\mathcal{L}_{\text{CE}} = - \sum_i y_i \log(\hat{y}_i) \quad (8)$$

During training, cross-entropy loss is used to compare the predicted token distribution with the actual target distribution. Minimizing this loss guides the model toward accurate code generation.

9. Backpropagation

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (9)$$

The chain rule is used to calculate gradients of the loss with respect to each trainable parameter in the network. This is the core idea behind backpropagation, enabling learning from data.

10. Gradient Descent Update

$$w := w - \eta \cdot \frac{\partial \mathcal{L}}{\partial w} \quad (10)$$

Model weights are updated by moving in the direction that minimizes the loss. Here, η is the learning rate that controls the step size of each update.

In summary, these mathematical operations form the backbone of the training pipeline in transformer-based LLMs. By fine-tuning such models on domain-specific prompts and aligning them with Manim-based code generation, our project leverages these principles to automate the transformation of mathematical descriptions into educational animations.

References

- [1] A. Arcavi, “The role of visual representations in the learning of mathematics,” *Educational Studies in Mathematics*, vol. 52, no. 3, pp. 215–241, 2003.
- [2] R. E. Mayer, “Cognitive theory of multimedia learning,” in *The Cambridge Handbook of Multimedia Learning*. Cambridge University Press, 2014, pp. 43–71.
- [3] 3Blue1Brown, “Manim documentation,” 2020, <https://docs.manim.community/en/stable/>.
- [4] ManimCommunity, “Manim,” 2023, <https://github.com/ManimCommunity/manim>.
- [5] A. Meurer, C. P. Smith, M. Paprocki *et al.*, “SymPy: symbolic computing in python,” *PeerJ Computer Science*, vol. 3, p. e103, 2017.
- [6] A. Radford, J. W. Kim, J. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” *arXiv preprint arXiv:2103.00020*, 2021.
- [7] OpenAI, “Openai codex,” 2021, <https://openai.com/blog/openai-codex>.
- [8] L. Tunstall, L. von Platen, and T. Wolf, *Natural Language Processing with Transformers*. O’Reilly Media, 2022.
- [9] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [10] M. AI, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [11] D. Hendrycks, S. Basart, C. Burns, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Natural language to code generation with transformers,” *arXiv preprint arXiv:2108.07732*, 2021.
- [12] Hugging Face, “Fine-tuning transformers,” 2023, <https://huggingface.co/docs/transformers/training>.
- [13] LangChain, “Natural language to python code,” 2023, <https://python.langchain.com>.
- [14] F. Khan, “Ai text to video model from scratch,” <https://github.com/FareedKhan-dev/AI-text-to-video-model-from-scratch>, 2024, gitHub repository.

- [15] F. Tan, S. Feng, and V. Ordonez, “Text2scene: Generating compositional scenes from textual descriptions,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 6710–6719.
- [16] OpenAI. (2024) Introducing sora: A text-to-video model. OpenAI. [Online; accessed 3-June-2025]. [Online]. Available: <https://openai.com/sora>
- [17] Google DeepMind. (2025) Veo: Our state-of-the-art video generation model. Google DeepMind. [Online; accessed 3-June-2025]. [Online]. Available: <https://deepmind.google/models/veo/>
- [18] Luma AI. (2024) Dream machine: Real-time text-to-video generation. Luma AI. [Online; accessed 3-June-2025].